

Weighted A* Search

Weighted A* is a variant of the standard A* search algorithm that introduces a trade-off between optimality and search efficiency. By weighting the heuristic function, Weighted A* can prioritize faster searches at the expense of finding the absolute shortest path. This approach is particularly useful in situations where finding a solution quickly is more important than finding the optimal solution.

Key Concepts

1. Heuristic Weighting:

- In standard A*, the evaluation function is $f(n)=g(n)+h(n)$, where $g(n)$ is the cost to reach node n from the start node, and $h(n)$ is the heuristic estimate of the cost from n to the goal.
- In Weighted A*, the evaluation function is modified to $f'(n)=g(n)+w \times h(n)$, where w is a weight factor greater than 1.
- The weight w amplifies the heuristic function, making the search algorithm more "greedy" by placing more emphasis on estimated distance to the goal rather than the cost incurred so far.

2. Trade-off Between Optimality and Speed:

- By increasing the weight w , Weighted A* focuses more on reaching the goal quickly, potentially at the expense of bypassing the optimal path.
- When $w=1$, Weighted A* is equivalent to standard A* and will find the optimal path if the heuristic is admissible.
- When $w>1$, the path found may be suboptimal, but the search may complete faster by exploring fewer nodes.

3. Bounded Suboptimality:

- Weighted A* guarantees that the cost of the path found will not exceed w times the cost of the optimal path. This means that the solution is within a known factor of the optimal solution, providing a controlled trade-off between search time and path quality.

A* search: $g(n) + h(n)$ ($W = 1$)

Uniform-cost search: $g(n)$ ($W = 0$)

Greedy best-first search: $h(n)$ ($W = \infty$)

Weighted A* search: $g(n) + W \times h(n)$ ($1 < W < \infty$)

How Weighted A* Works

1. Initialization:

- Begin with the start node S and calculate its heuristic estimate $h(S)$.
- Set the initial threshold as $f'(S) = g(S) + w \times h(S)$.

2. Node Expansion:

- Similar to standard A*, expand the node with the lowest $f'(n)$ value, where $f'(n) = g(n) + w \times h(n)$.
- Add the successors of the current node to the open list (frontier).

3. Goal Test:

- If the goal node G is reached, the search stops, and the path from S to G is reconstructed.

4. Termination:

- The algorithm continues expanding nodes until the goal is reached or no more nodes can be expanded.

Advantages of Weighted A*

1. **Faster Search:**

- By increasing the weight w , Weighted A* can reach the goal faster, making it particularly useful in real-time applications or when quick decisions are required.

2. **Controlled Suboptimality:**

- Weighted A* provides a way to control the trade-off between search speed and path optimality. By adjusting the weight w , users can decide how much they are willing to compromise on path quality to gain speed.

3. **Flexibility:**

- The ability to tune the weight w allows Weighted A* to be adapted to different scenarios, balancing between speed and optimality depending on the needs of the application.

4. **Same Implementation Framework:**

- Weighted A* can be implemented using the same framework as standard A*, with minimal modifications, making it easy to integrate into existing systems.

Limitations of Weighted A*

Suboptimal Paths:

- The primary drawback of Weighted A* is that it may produce suboptimal paths, especially as w increases. This can be problematic in applications where optimality is crucial.

Heuristic Dependence:

- The effectiveness of Weighted A* heavily depends on the quality of the heuristic. If the heuristic is poor, increasing the weight might lead to exploring misleading paths, reducing the overall efficiency.

Bounding Factor:

- While the path cost is bounded by $w \times$ optimal cost, in practice, the actual suboptimality might be closer to this upper bound, particularly with high weights.

Risk of Missing Optimal Paths:

- In some search spaces, emphasizing the heuristic too much might cause Weighted A* to miss shorter but less obvious paths, leading to significantly suboptimal solutions.

Real-life applications of A* search

Air Traffic Management:

- **Application:** A* is used in air traffic management systems to optimize flight paths, helping avoid collisions and manage airspace efficiently.
- **Example:** A* can help in planning optimal flight paths for aircraft to avoid no-fly zones, minimize fuel consumption, and ensure timely arrivals.

Autonomous Vehicles:

- **Application:** A* is used in the path planning systems of autonomous vehicles, helping them navigate safely through urban environments.
- **Example:** Self-driving cars use A* to plan routes that avoid obstacles, follow traffic rules, and adapt to changing road conditions. time.

Natural Language Processing (NLP):

- **Application:** A* can be used in NLP tasks that require searching through possible sequences, such as in parsing sentences or finding the most likely sequence of words.
- **Example:** In speech recognition, A* can help find the most likely word sequence that matches the given sound input.

Virtual Assistants and Chatbots:

- **Application:** A* can be employed in virtual assistants and chatbots to determine the best sequence of actions or responses based on user input and context.
- **Example:** A chatbot may use A* to determine the most appropriate sequence of responses to guide a user through a troubleshooting process or a decision-making tree.

Medical Diagnosis Systems:

- **Application:** A* can be used in decision support systems for medical diagnosis, helping to find the most probable diagnosis or treatment path based on symptoms and medical history.
- **Example:** A medical expert system might use A* to suggest a series of diagnostic tests that lead to an accurate diagnosis in the shortest possible time.

Recursive Best-First Search

Recursive Best-First Search (RBFS) is an informed search algorithm that combines the benefits of best-first search and depth-first search while maintaining a low memory footprint. It is designed to address the memory limitations of traditional best-first search algorithms, such as A*, by using recursion and a limited memory strategy to explore paths.

Key Features of Recursive Best-First Search

1. **Memory Efficiency:** RBFS uses memory more efficiently than traditional best-first search algorithms like A*, as it only retains a single path from the root to the current node, along with some additional information about the best alternative paths.
2. **Optimality:** RBFS can be optimal if it uses an admissible heuristic function (i.e., one that does not overestimate the cost to reach the goal).
3. **Completeness:** RBFS is complete; it will find a solution if one exists in a finite search space.
4. **Recursive Nature:** The algorithm is inherently recursive, simplifying its implementation and reducing the need for explicit stack management.

How Recursive Best-First Search Works

RBFS operates using a recursive function that expands nodes based on their estimated cost while managing a limited memory footprint. The main components of the algorithm include:

1. Evaluation Function:

- Similar to A* and other best-first search algorithms, RBFS uses an evaluation function:

$$f(n)=g(n)+h(n)$$

- Where:
 - $g(n)$: The cost from the start node to node n .
 - $h(n)$: The heuristic estimate of the cost from node n to the goal.

2. Recursive Search:

- RBFS recursively explores nodes by keeping track of the current node's cost and the best alternative path cost.
- The recursive function calls itself to explore child nodes, maintaining the path from the start to the current node.

3. Path and Cost Management:

- Each recursive call keeps track of the current best path and its cost. If the best path cost exceeds a threshold, the algorithm backtracks and explores alternative paths.

4. Backtracking:

- If a leaf node is reached or a node's $f(n)$ exceeds the current best cost, the algorithm backtracks to explore other nodes, updating the best cost as necessary.

Steps of Recursive Best-First Search

1. Initialization:

- Start with the initial node (the starting state).
- Set an initial threshold based on the heuristic value $f(\text{start})=h(\text{start})$.

2. Recursive Exploration:

- Call the recursive function with the current node:
 - Expand the node by generating its successors.
 - For each successor, calculate its $f(n)$ value.
 - If a successor is the goal, return the path and cost.
 - If the best successor's $f(n)$ exceeds the threshold, backtrack and update the threshold.

3. Threshold Management:

- Keep track of the lowest $f(n)$ value among all successors.
- If a better path is found, update the best path and continue searching.

4. Repeat:

- Continue the recursive process until a solution is found or all nodes have been explored.

Properties of Recursive Best-First Search

1. **Optimality:**
 - RBFS guarantees optimality if the heuristic is admissible and the cost function is consistent.
2. **Completeness:**
 - The algorithm is complete, meaning it will find a solution if one exists.
3. **Memory Usage:**
 - RBFS uses less memory compared to traditional best-first search algorithms because it does not store all nodes, just the current path and some additional information.
4. **Performance:**
 - The performance of RBFS can be comparable to A* in some scenarios, particularly when the heuristic is well-designed.

Advantages of Recursive Best-First Search

1. **Low Memory Usage:** RBFS is particularly useful in situations where memory is a constraint, as it only retains the current path.
2. **Simple Implementation:** The recursive nature of RBFS simplifies the implementation of the algorithm, avoiding explicit stack management.
3. **Flexible Heuristic Use:** RBFS can be adapted with different heuristics to fit various problems.

Disadvantages of Recursive Best-First Search

1. **Repeated Node Expansions:** RBFS may explore some nodes multiple times, especially in cases where paths are revisited.
2. **Time Complexity:** While RBFS uses less memory, it may take longer to find solutions due to its recursive nature and potential repeated explorations.
3. **Dependence on Heuristic Quality:** The efficiency of RBFS heavily relies on the quality of the heuristic function used.

Memory Bounded A* vs RBFS

Memory Management:

- **SMA***: Operates within a fixed memory limit, pruning least promising nodes when memory is full.
- **RBFS**: Uses linear memory relative to the search depth, only keeps current path and best alternative path.

Node Pruning:

- **SMA***: Prunes nodes when memory is full and stores the best f-value of pruned nodes for possible regeneration.
- **RBFS**: Does not prune; instead, it discards all but the current path and the best alternative, leading to potential re-expansion of nodes.

Backtracking:

- **SMA***: Backtracks by re-expanding pruned nodes based on stored f-values, balancing memory use with exploration.
- **RBFS**: Backtracks by revisiting the best alternative path, potentially causing redundant re-expansions without memory of previously explored paths.

Optimality:

- **SMA***: Attempts to maintain optimality within memory limits but may sacrifice optimality if key nodes are pruned.
- **RBFS**: Can find optimal solutions if the heuristic is admissible, but frequent backtracking might lead to inefficiency.

Efficiency:

- **SMA***: More efficient in large, complex spaces due to its memory management strategy, which reduces unnecessary re-expansions.
- **RBFS**: Less efficient due to potential repeated work during backtracking, as it lacks memory of pruned nodes.

Implementation Complexity:

- **SMA***: More complex due to the need for memory management and node regeneration mechanisms.
- **RBFS**: Simpler to implement as it primarily focuses on depth-first search with limited memory overhead.

Beam Search

Beam Search is a heuristic search algorithm that is often used in scenarios where the search space is vast, such as natural language processing (NLP), machine translation, speech recognition, and other AI applications. It is a breadth-first search algorithm but with a limited memory footprint. Instead of exploring all possible paths like in standard breadth-first search, Beam Search keeps track of only the most promising paths at each level of the search tree, defined by a fixed beam width.

Key Concepts

1. **Beam Width:**

- The beam width is a parameter that determines the number of top candidates (nodes) that the algorithm keeps at each level of the search tree.
- For example, if the beam width is set to 3, Beam Search will keep only the top 3 most promising paths at each level.

2. **Breadth-First Search with Pruning:**

- Beam Search explores the search tree level by level, similar to breadth-first search.
- However, instead of keeping all nodes at each level, it only retains a fixed number (equal to the beam width) of the most promising nodes, discarding the rest.

3. **Evaluation Function:**

- The algorithm uses an evaluation function (often similar to the one used in A*) to rank the nodes. Typically, the evaluation function is $f(n)=g(n)+h(n)$, where:
 - $g(n)$ is the cost to reach node n .
 - $h(n)$ is the heuristic estimate of the cost from n to the goal.
- In cases like language generation, the evaluation function might be based on the likelihood of generating the next word or token.

4. **Pruning Less Promising Paths:**

- At each level of the search tree, Beam Search selects the top candidates based on the evaluation function.
- Paths that do not rank within the top k (where k is the beam width) are discarded.

5. **Termination:**

- The search continues until it reaches a predefined stopping condition, such as finding a goal state, reaching a maximum depth, or completing a fixed number of iterations.

How Beam Search Works

1. **Initialization:**
 - Start with the initial node, often called the root node. The algorithm calculates its evaluation function and adds it to the list of active candidates.
2. **Expand Candidates:**
 - From the list of current candidates (initially just the root node), expand each candidate by generating its successors (children nodes).
3. **Evaluate and Prune:**
 - For each successor, calculate the evaluation function.
 - Sort the successors based on their evaluation scores.
 - Select the top k (beam width) successors and prune the rest.
4. **Repeat:**
 - Treat the selected successors as the new list of current candidates and repeat the process of expansion, evaluation, and pruning.
5. **Termination:**
 - The algorithm continues this process until it either reaches the goal state or satisfies another termination condition (e.g., reaching a certain depth in the search tree).

Key Characteristics and Use Cases

Memory Efficiency:

- Beam Search is more memory-efficient than exhaustive search methods like breadth-first search because it only retains a fixed number of paths (equal to the beam width) at each level.

Scalability:

- Beam Search scales well in large search spaces, such as language generation or sequence prediction tasks, because it limits the number of paths explored.

Controlled Complexity:

- The beam width k allows control over the trade-off between search thoroughness and computational complexity. A larger beam width means more paths are considered, increasing the chances of finding a better solution but at the cost of higher computational resources.

Differences from Other Search Algorithms

- **Beam Search vs. Breadth-First Search:**
 - **Breadth-First Search:** Explores all nodes at each level of the search tree, leading to high memory usage.
 - **Beam Search:** Prunes all but the top k nodes at each level, reducing memory usage and making the search more manageable in large spaces.
- **Beam Search vs. A*:**
 - **A*:** Keeps all nodes in memory and always expands the most promising node based on a combination of path cost and heuristic (optimal but memory-intensive).
 - **Beam Search:** Focuses on keeping only a fixed number of promising paths, sacrificing optimality for efficiency.
- **Beam Search vs. Greedy Best-First Search:**
 - **Greedy Best-First Search:** Expands nodes based solely on the heuristic, potentially ignoring the true cost to reach a node.
 - **Beam Search:** Can be configured to balance between the heuristic and the path cost, but with a focus on maintaining a limited set of candidates.

Limitations

1. **Suboptimal Solutions:**

- Beam Search is not guaranteed to find the optimal solution because it might prune paths that could lead to a better outcome later on.

2. **Heuristic Dependence:**

- The performance of Beam Search heavily depends on the quality of the evaluation function. A poorly designed heuristic can lead to suboptimal or even incorrect solutions.

3. **Limited Exploration:**

- By restricting the number of paths considered at each step, Beam Search may miss better solutions that would be found with a broader or exhaustive search.

Applications

- 1. Machine Translation:** Beam Search is a core component in sequence-to-sequence models used in machine translation systems, such as Google Translate. When translating a sentence from one language to another, the model generates possible translations for each word or phrase, and Beam Search is used to explore the most promising translation paths.
- 2. Speech Recognition:** In speech recognition systems like those used by virtual assistants (e.g., Siri, Google Assistant), Beam Search is used to find the most likely sequence of words given a sequence of audio inputs. The algorithm processes the audio data to generate possible word sequences, then uses Beam Search to select the most likely sequence based on a language model.
- 3. Text Generation and Summarization:** In generative models like GPT, Beam Search is used to generate coherent and contextually appropriate text by selecting the best sequence of words or tokens. This is also applied in automatic text summarization, where the system generates a concise summary from a longer document.
- 4. Path Planning in Robotics:** Beam Search is used in robotics for path planning, where a robot must navigate from a start position to a goal while avoiding obstacles. The algorithm evaluates multiple possible paths at each step and selects the most promising ones to explore further.
- 5. DNA Sequence Alignment:** In bioinformatics, Beam Search is used for aligning DNA sequences to identify regions of similarity, which can indicate functional, structural, or evolutionary relationships between sequences.
- 6. Dialogue Systems and Chatbots:** In dialogue systems, Beam Search is used to generate the most appropriate response in a conversation by exploring multiple possible replies and selecting the one that best matches the context of the conversation.

Summary of Benefits Over Other Search Algorithms:

- **Efficiency:** Beam Search is more efficient than exhaustive search methods because it prunes less promising paths early, reducing computational complexity.
- **Memory Usage:** It uses significantly less memory than algorithms like A* since it only retains a fixed number of paths (defined by the beam width) at each level of the search tree.
- **Quality:** While it may sacrifice some optimality compared to exhaustive searches, Beam Search often provides better results than greedy algorithms by considering multiple possibilities at each step.
- **Scalability:** Beam Search is scalable to large problems, making it suitable for real-world applications where search spaces can be vast, such as language processing, speech recognition, and robotics.

Admissible Heuristics: An Example

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

$h_1(n)$ = number of misplaced tiles

$h_1(S) = ?$

$h_1(S) = 8$

$h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)

$h_2(S) = ?$

$h_2(S) = 3+1+2+2+2+3+3+2 = 18$

Heuristic Property: Dominance

Dominance occurs when one heuristic is consistently more effective than another in estimating the true cost to reach the goal. Specifically, a heuristic h_1 is said to **dominate** another heuristic h_2 if, for every possible state n , the estimate provided by $h_1(n)$ is closer to the actual cost $h^*(n)$ of reaching the goal than the estimate provided by $h_2(n)$.

Mathematically, this can be expressed as: $h_1(n) \geq h_2(n)$ for all states n , and $h_1(n)$ is closer to $h^*(n)$ than $h_2(n)$.

Dominance and Admissibility:

1. **Admissibility** of a heuristic means that the heuristic never overestimates the true cost to reach the goal (i.e., $h(n) \leq h^*(n)$ for all n).
 - If h_1 dominates h_2 , and both heuristics are admissible, then h_1 is guaranteed to be more effective in terms of the search process, as it will guide the search more directly towards the goal, potentially expanding fewer nodes.
2. **Example of Dominance:**
 - **Heuristic Example:**
 - Suppose we have two heuristics for a pathfinding problem on a grid:
 - $h_1(n)$ = straight-line distance from the current node to the goal.
 - $h_2(n)$ = Manhattan distance (the sum of the absolute differences in the horizontal and vertical distances to the goal).
 - In this case, h_1 might dominate h_2 because the straight-line distance is always equal to or greater than the Manhattan distance (assuming all movements cost the same). Thus, h_1 gives a more accurate estimate of the true cost in a direct path scenario.

○

Why Dominance Matters

- **Search Efficiency:**
 - Using a dominant heuristic can significantly reduce the number of nodes explored during the search process, leading to faster solution times and less computational effort.
- **Algorithm Performance:**
 - In practice, dominance is an essential property to consider when choosing or designing heuristics for a particular problem. A heuristic that dominates others can make the difference between a search algorithm that performs efficiently and one that is prohibitively slow.
- **Combining Heuristics:**
 - In some cases, if h_1 dominates h_2 , a combined heuristic such as $h(n) = \max(h_1(n), h_2(n))$ can be used, which still preserves admissibility while benefiting from the best features of both heuristics.

Relaxed Problems

Relaxed Problem:

- A relaxed problem is a modified version of an original problem where some of the constraints are either loosened or removed entirely. The relaxation typically results in a problem that is easier to solve or analyze.
- The solution to the relaxed problem provides an estimate or a bound that can be used as a heuristic in solving the original, more complex problem.

They are critically important in computing admissible heuristics. Why?

Purpose of Relaxation:

- The primary purpose of creating a relaxed problem is to derive a heuristic that can guide a search algorithm efficiently towards the solution of the original problem.
- Relaxed problems are used to generate admissible heuristics, which never overestimate the true cost of reaching the goal in the original problem.

Example: Imagine we have to plan the best route for a delivery truck that has to stop at many different places. The original problem is very complex because there are a lot of rules to follow, like which roads are open, traffic conditions, and the order of deliveries.

- Now, let's say we simplify the problem by ignoring some of those rules. For example, you pretend that there's no traffic or that the truck can take any road, even if it's closed. This simplified version of the problem is what we call a "relaxed problem."
- When you solve this relaxed problem, you're essentially looking at all possible ways to solve the puzzle, including some that wouldn't work in the real world. Because you're considering more options, the solution you find in this relaxed world will either be cheaper (in terms of time, distance, or cost) or at least no more expensive than the best possible solution to the original, harder problem.
- This is useful because the relaxed problem can give you a good estimate or starting point for solving the original problem. It helps you figure out a strategy that might not be perfect but gets you close to the best solution faster.

8 Puzzle: Relaxation

The 8-Puzzle Problem:

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

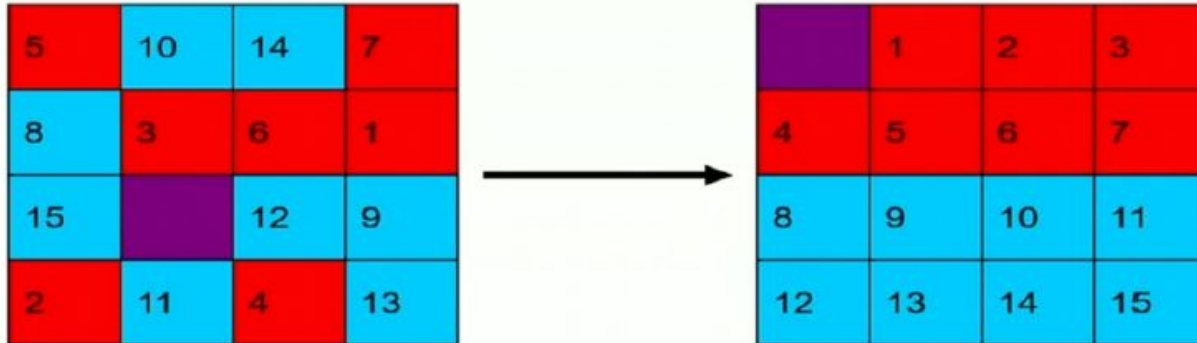
- **Original Problem:** Constraints
 - a. Swap with the space
 - b. Swap with the neighbour
- **Relaxation:**
 - a. For h_1 , I remove the 2nd constraint i.e., swap with neighbour but keep the 1st one that it can only swap with gaps, then I am allowing the tiles to fly.
 - i. So here my total cost will be 8, which is the cost of h_1 hence, we find h_1 to be an admissible heuristic.
 - b. For h_2 , I relax the constraint of swapping again and allow the tiles to move over one another.
 - i. Eg., allow 4 to share the tile with 6 before moving to its actual position i.e., the gap.
- If the rules of the 8-puzzle are relaxed so that a tile can move anywhere, then $h_1(n)$ gives the shortest solution.
- If the rules are relaxed so that a tile can move to any adjacent square, then $h_2(n)$ gives the shortest solution.

Pattern Database Heuristics

Pattern Database Heuristics (PDBs) are a specific type of heuristic used in search algorithms, particularly in solving complex combinatorial problems like sliding tile puzzles, Rubik's Cube, and other similar problems. The core idea behind pattern database heuristics is to precompute and store the exact solutions for simplified versions of the problem—referred to as "patterns"—and then use these precomputed solutions as heuristics during the actual search process.

Definition of Pattern Database Heuristics

- **Pattern:**
 - A pattern in this context refers to a subset of the problem's components (e.g., a subset of tiles in a puzzle) while ignoring the rest. The pattern represents a smaller, simpler version of the original problem.
- **Pattern Database (PDB):**
 - A pattern database is a large lookup table that stores the optimal solution (e.g., minimum number of moves) for all possible configurations of a given pattern.
 - This database is created by exhaustively solving the subproblem represented by the pattern and recording the cost to reach the goal configuration for each possible state of the pattern.
- **Heuristic Calculation:**
 - During the actual search in the original problem, the current state is matched against the patterns. The heuristic value is obtained by looking up the pattern database to get the precomputed cost for that pattern, which gives an estimate of how far the problem is from the goal.



Advantages:

- **Efficiency:** Pattern database heuristics can significantly speed up the search process by providing accurate, precomputed estimates that reduce the number of states the algorithm needs to explore.
- **Scalability:** For large and complex problems, where calculating exact solutions on-the-fly would be computationally expensive, pattern database heuristics offer a practical way to handle the complexity.
- **Improved Search Performance:** When integrated with search algorithms like A*, pattern database heuristics can dramatically reduce the search space, making it feasible to solve problems that would otherwise be intractable.

Disadvantages:

- **Memory Usage:** Storing the pattern database can require significant amounts of memory, especially for larger patterns or more complex problems.
- **Precomputation Time:** Creating the pattern database requires an exhaustive search of all possible configurations of the pattern, which can be time-consuming.
- **Complexity of Design:** Designing effective pattern databases involves carefully choosing patterns that provide meaningful heuristics while balancing memory usage and computational complexity.

Search In Complex Environments

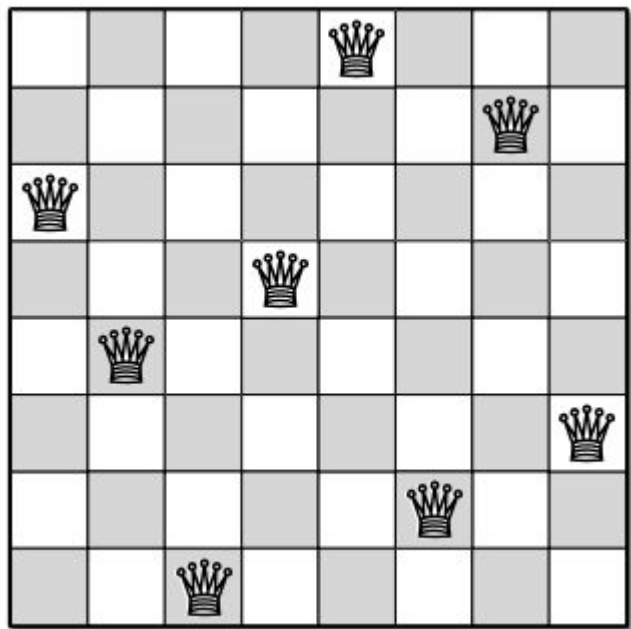
The slide features a light blue background with decorative elements. At the top, there are two horizontal lines: a thick teal line followed by a thin light blue line. At the bottom, there are two horizontal lines: a thin light blue line followed by a thick teal line. Two short, thick, olive-green horizontal dashes are positioned on the left and right sides of the slide, centered vertically between the top and bottom line sets.

Local Search and Optimization

General Idea

- When we think of the word local search in the modern-day age, we think about something like this, right.
 - Tell me all restaurants near IIT right, or consult Google first
 - Local search, as it says, is any search aimed at finding something within a specific geographical area.
- General search algorithms: Any model can be, any problem can be formulated as a search problem and is solved using one algorithm, one search algorithm called the Local search algorithm. It is a series, a set of algorithms.
- There are some differences in the way we have formulated any given problem earlier, and how we are going to formulate it now.
- Previous lectures:
 - Path to goal is solution to problem,
 - we had start state, goal state, objective was to find what sequence of actions to apply
 - State/node was some partial information of the search so far
 - Systematic exploration of search space.
- From now on:
 - We will now be in a setting where solution is not the path but the state itself.
 - In other words, we will search in solution space.
 - Each search node would be a good/bad solution

N Queens Problem



The **N-Queens problem** is a classic combinatorial puzzle where the objective is to place **N queens** on an **N x N chessboard** in such a way that no two queens threaten each other. This means no two queens can share the same row, column, or diagonal. The challenge is to find all valid configurations or a specific solution where these conditions are met.

Initial State: An empty chessboard (no queens placed yet), or a partial configuration with some queens already placed in the first few columns.

Goal State: A configuration where N queens are placed on the board such that no two queens threaten each other (i.e., no two queens share the same row, column, or diagonal).

Action: Add one queen.

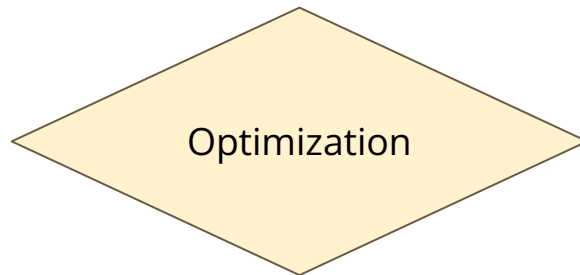
Search Algorithms: Informed and Uninformed

- Trying to find the shortest path to the goal.

Now, in general we are interested in the final state, satisfying all the constraints.

This can be modelled as a Local search problem.

Satisfaction vs. Optimization



- Up until now, we had been looking at satisfaction problems. Satisfaction problem is to find the path to the goal, and once we find the path, we want the best path to the goal.
- So, there is some optimisation going also.
- But suppose we are only interested in finding a path to the goal. That is what we are going to call a satisfaction problem.
- Optimisation problems, is like finding the optimal path to optimising the objective function in general.
- In the satisfaction world, we have to find a solution that satisfies all constraints.
- In the optimization world, we have an objective function and want to minimise or maximise.
- In satisfaction, I have given just several constraints. In Optimisation, I have given the number of constraints and an objective function.

Example



- N Queens problem is a satisfaction problem or an optimization problem?
- It is a satisfaction problem, but we will formulate it as an optimization problem now, and a state would be a solution.
- We have to develop an objective function that is analogous to the original objective function of the satisfaction problem when optimized fully.

Optimization Problems

Pure Optimization Problems

- **Definition:**
 - In a pure optimization problem, the objective is to find the best possible state (or configuration) that maximizes or minimizes a given objective function. The problem is usually defined in terms of:
 - **Objective Function:** A function that assigns a value (fitness, cost, utility, etc.) to each possible state. The goal is to optimize this function.
 - **State Space:** All possible configurations or states that the variables of the problem can take.
 - **Constraints:** Conditions that the solutions must satisfy (in constrained optimization problems).
- **No Goal Test or Path Cost:**
 - Unlike "standard" search problems (like those introduced in previous slides), optimization problems often do not have a **goal test**—there is no predefined target state to reach. Instead, the search is about finding the best state according to the objective function.
 - Additionally, there is typically no **path cost** associated with reaching the solution. The focus is on the quality of the final state, not on how it was reached.

Examples of Pure Optimization Problems

- **Engineering Design:** Finding the optimal design parameters for a bridge that minimizes cost while maximizing strength.
- **Portfolio Optimization:** Selecting a combination of financial assets that maximizes return for a given level of risk.
- **Traveling Salesperson Problem (TSP):** Finding the shortest possible route that visits each city exactly once and returns to the origin city.
- **Machine Learning Model Tuning:** Selecting hyperparameters that minimize the error on validation data.

Technical Description: Local Search

Local search is a type of optimization technique used to solve complex computational problems, particularly in large and combinatorial search spaces where finding an exact solution might be computationally infeasible. Instead of exhaustively exploring the entire search space, local search focuses on iteratively improving a single solution by making small, incremental changes, often referred to as "moves." The goal is to find a solution that is better than the current one by exploring its "neighborhood."

1. Key Concepts of Local Search

1. **Search Space:**
 - The search space represents all possible solutions to a problem. In local search, the focus is on exploring a small portion of this space around a current solution rather than attempting to traverse the entire space.
2. **Current Solution:**
 - Local search begins with an initial solution, which can be generated randomly or through some heuristic method. This solution is not necessarily optimal but serves as a starting point.

3. Neighborhood:

- The neighborhood of a solution consists of all possible solutions that can be reached by making a small, defined change to the current solution. The exact nature of this change depends on the problem being solved. For example, in a traveling salesperson problem (TSP), the neighborhood might include all possible routes that can be created by swapping the order of two cities.

4. Objective Function:

- The objective function measures the quality or "fitness" of a solution. Local search algorithms aim to optimize this function, either by maximizing or minimizing its value, depending on the problem.

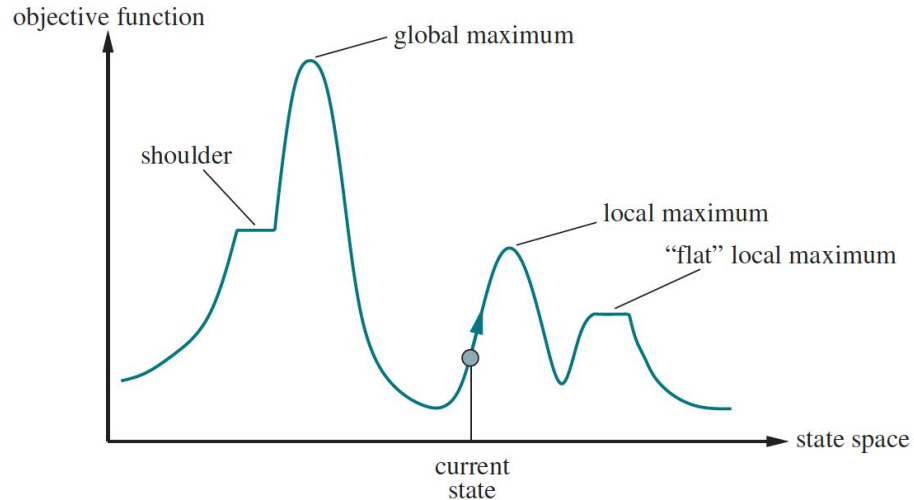
5. Move:

- A move refers to the transition from one solution to another within its neighborhood. This is achieved by making a small change to the current solution.

6 Termination Criteria:

- Local search algorithms typically terminate when a predefined condition is met, such as reaching a maximum number of iterations, achieving a solution of acceptable quality, or when no further improvement is possible (a local optimum is reached).

State-Space Landscape



A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum

Objective Function: The vertical axis represents the **objective function**, which could be a cost or reward function depending on the problem. The goal of optimization is to maximize or minimize this function.

State Space: The horizontal axis represents the **state space**, which consists of possible solutions or configurations in the problem domain.

Global Maximum: This is the highest point in the search space, representing the **best possible solution**. The goal of many optimization algorithms is to reach this point.

Local Maximum: A **local maximum** is a point where the objective function reaches a peak, but it is **not the highest point** in the entire search space. The algorithm can get stuck here, thinking it has found the best solution when it hasn't.

"Flat" Local Maximum: This is a region where the objective function is **flat** or constant over multiple states. It can prevent the algorithm from making progress because it doesn't have a clear direction to improve the solution.

Shoulder: The shoulder is a relatively flat area on the way to the global maximum. This flatness can temporarily deceive the algorithm into thinking it's reached a local maximum, though improvement may still be possible.

Current State: The arrow starting from the **current state** shows the algorithm's position in the search space. The image suggests that the algorithm is currently at a local maximum or is stuck on a flat region, potentially preventing it from reaching the global maximum.

Types of Local Search Algorithms

1. **Hill Climbing**
2. **Simulated Annealing**
3. **Tabu Search**
4. **Genetic Algorithms (GA)**
5. **Local Beam Search**
6. **Iterated Local Search**

Advantages of Local Search

- **Scalability:** Local search methods can handle very large search spaces because they focus on a small neighborhood around the current solution rather than attempting to explore the entire space.
- **Simplicity:** Many local search algorithms, like hill climbing, are conceptually simple and easy to implement.
- **Flexibility:** Local search can be applied to a wide range of problems, including those where other algorithms may be impractical due to the size or complexity of the search space.
- **Efficiency:** For many practical problems, local search can quickly find good (if not optimal) solutions, making it useful in real-time applications where time is a critical factor.

Limitations of Local Search

Local Optima: Local search algorithms can get stuck in local optima—solutions that are better than neighboring solutions but not the best overall. Overcoming this requires additional strategies, like simulated annealing or tabu search.

No Guarantee of Global Optimality: Because local search focuses on improving a single solution incrementally, it does not guarantee finding the global optimum unless the problem has a particular structure that allows it.

Problem-Specific Design: The effectiveness of a local search algorithm often depends on the careful design of the neighborhood structure and move operators, which can be problem-specific.

No Full Exploration: Local search does not explore the entire search space, so it might miss better solutions located far from the current solution.

Summary

- Local Search
 - Keep track of single current state
 - Move only to neighbouring states
 - Ignore paths
- “Pure Optimization” Problems
 - All states have an objective function
 - Goal is to find state with max (or min) objective value
 - Does not quite fit into path-cost/ goal-state formulation
 - Local search can do quite well on these problems.

Hill Climbing

Hill Climbing is a type of **local search algorithm** used for solving **optimization problems**, where the goal is to find the best solution according to an objective function. It is particularly useful for problems where the search space is vast and traditional exhaustive search methods are computationally infeasible.

- **Local Search Algorithm:**

- In contrast to global search algorithms, which attempt to explore the entire search space, local search algorithms focus on finding solutions by exploring the local neighborhood of a given solution. The term "local" implies that the algorithm's perspective is limited to the immediate vicinity of the current solution.

- **Mathematical Optimization:**

- Hill climbing is used in various mathematical optimization problems, where the objective is to either maximize or minimize a specific function. These problems can range from simple linear functions to complex, multi-dimensional spaces.

- **Iterative Process:**

- Hill climbing starts with an arbitrary solution, which can be chosen randomly or based on a heuristic. From this initial solution, the algorithm iteratively makes small changes, or "moves," to the solution. Each move involves evaluating the neighboring solutions (states) and selecting the one that improves the objective function the most. This process continues until no better neighboring solution can be found.

- **Neighboring Solutions:**

- A neighboring solution is one that is reachable from the current solution by making a small change. For example, in the case of a numerical optimization problem, a neighboring solution could be obtained by slightly increasing or decreasing the value of a variable.

Basic Idea of Hill Climbing

The name "hill climbing" comes from the metaphor of climbing a hill: just as a climber takes steps in the direction that leads upward to reach the top of a hill, the algorithm takes steps in the direction that improves the objective function. The "hill" in this metaphor represents the landscape of the objective function, where higher points correspond to better (higher) values of the function in the case of maximization, and lower points correspond to better (lower) values in the case of minimization.

- **Direction of Movement:**
 - **Maximization Problems:** In these problems, the goal is to find the maximum value of the objective function. The algorithm moves in the direction of increasing the function's value, akin to climbing up a hill.
 - **Minimization Problems:** Here, the goal is to find the minimum value of the objective function. The algorithm moves in the direction of decreasing the function's value, which can be thought of as descending a hill or valley to reach the lowest point.
- **Local Optimum:**
 - The algorithm stops when it reaches a point where no neighboring solution has a better value than the current solution. This point is called a **local optimum**. It is important to note that a local optimum is not necessarily the best possible solution (global optimum) but is the best in the immediate vicinity of the current solution.
- **No Exploration of the Entire Search Space:**
 - Hill climbing does not attempt to explore the entire search space. Instead, it focuses on improving the current solution by evaluating and selecting among its neighbors. This makes the algorithm efficient in terms of time and space, as it only needs to consider a small portion of the search space at each step.

Hill Climbing Algorithm

1. Start with an Initial Solution

- **Objective:** Establish a starting point for the algorithm.
- **Process:**
 - **Initial State Selection:**
 - The algorithm begins by choosing an initial state or solution. This initial state can be selected in various ways:
 - **Random Selection:** In many cases, the initial solution is chosen randomly from the search space. This randomness helps ensure that the algorithm starts from a diverse range of points if multiple runs are performed.
 - **Heuristic-Based Selection:** Alternatively, the initial solution can be chosen based on some heuristic that suggests a potentially good starting point. For example, in a scheduling problem, you might start with a schedule that you know satisfies some key constraints.
- **Importance:**
 - The choice of the initial solution can significantly impact the algorithm's performance. A good initial solution may lead to quicker convergence to an optimal or near-optimal solution, while a poor initial choice might result in the algorithm getting stuck in suboptimal areas of the search space.

2. Evaluate the Objective Function

- **Objective:** Measure the quality of the current solution.
- **Process:**
 - **Objective Function Computation:**
 - The algorithm evaluates the objective function at the current solution. The objective function is a mathematical expression that quantifies how good or bad a solution is with respect to the problem being solved.
 - **For Maximization Problems:** The objective function's value is higher for better solutions. The goal is to maximize this value.
 - **For Minimization Problems:** The objective function's value is lower for better solutions. The goal is to minimize this value.
- **Importance:**
 - The objective function is central to the hill climbing algorithm. It determines the direction of the search by guiding the algorithm towards better solutions.
- **Example:**
 - If the problem is to maximize a function $f(x) = -x^2 + 4x$, and the initial solution is $x=1$, the objective function value would be $f(1) = -1^2 + 4(1) = 3$.

3. Generate Neighboring Solutions

- **Objective:** Explore the local area around the current solution to find potential improvements.
- **Process:**
 - **Defining the Neighborhood:**
 - The algorithm identifies all neighboring solutions. A neighbor is a solution that can be reached from the current solution by making a small, defined change.
 - **Neighborhood Definition:** The definition of a neighborhood depends on the problem. For example:
 - **In a Numerical Optimization Problem:** A neighbor might be a solution obtained by slightly increasing or decreasing the value of a variable.
 - **In a Combinatorial Problem (e.g., TSP):** A neighbor might be generated by swapping two cities in the tour.
- **Importance:**
 - The size and structure of the neighborhood significantly impact the algorithm's effectiveness. A well-defined neighborhood ensures that the algorithm can explore the search space effectively and avoid getting stuck in suboptimal regions.
- **Example:**
 - For $x=1$, possible neighbors might be $x=0.9$ and $x=1.1$. The objective function values at these neighbors would be computed next.

4. Select the Best Neighbor

- **Objective:** Choose the most promising neighboring solution.
- **Process:**
 - **Evaluate Neighboring Solutions:**
 - The algorithm computes the objective function values for all the neighboring solutions identified in the previous step.
 - **Best Neighbor Selection:**
 - Among the neighboring solutions, the one that offers the greatest improvement in the objective function is selected. This move ensures that the algorithm is always progressing towards a better solution.
 - **For Maximization Problems:** The neighbor with the highest objective function value is chosen.
 - **For Minimization Problems:** The neighbor with the lowest objective function value is chosen.
- **Importance:**
 - This step ensures that the algorithm makes the best possible move at each iteration, steadily improving the solution. However, this greedy approach can sometimes lead the algorithm to get stuck in local optima.
- **Example:**
 - If $x = 0.9$ yields a value of 2.81 and $x = 1.1$ yields a value of 2.99, the algorithm selects $x = 1.1$ as the next solution because it has the higher value.

5. Move to the Neighboring Solution

- **Objective:** Update the current solution to the selected neighbor.
- **Process:**
 - **Transition to the New Solution:**
 - The algorithm moves to the best neighboring solution selected in the previous step. This solution becomes the new "current solution" for the next iteration.
 - **Repeat the Process:**
 - The algorithm then repeats the process, starting from evaluating the objective function at the new current solution, generating its neighbors, and selecting the best one.
- **Importance:**
 - This step ensures the algorithm is always progressing towards better solutions, provided such solutions exist in the neighborhood.
- **Example:**
 - The algorithm updates the current solution from $x=1$ to $x=1.1$ and repeats the process from this new starting point.

6. Termination Criteria

- **Objective:** Determine when the algorithm should stop running.
- **Process:**
 - **No Improvement Found:**
 - The most common termination criterion is when no neighboring solution improves the objective function. This situation indicates that the algorithm has reached a **local optimum**—a solution that is better than all its neighbors but may not be the best overall solution (global optimum).
 - **Other Termination Criteria:**
 - **Maximum Iterations:** The algorithm stops after a predefined number of iterations, regardless of whether an optimal solution has been found.
 - **Threshold Value:** The algorithm stops if the objective function value reaches or exceeds (for maximization) or falls below (for minimization) a certain threshold.
 - **Time Limit:** The algorithm stops after running for a certain amount of time.
- **Importance:**
 - Proper termination criteria are crucial to ensure that the algorithm does not run indefinitely and that it finds a solution in a reasonable time. However, it also determines how close the algorithm gets to the global optimum.
- **Example:**
 - If the algorithm reaches a point where increasing or decreasing x no longer improves the objective function, it stops and returns the current solution as the final result.

Algorithm

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

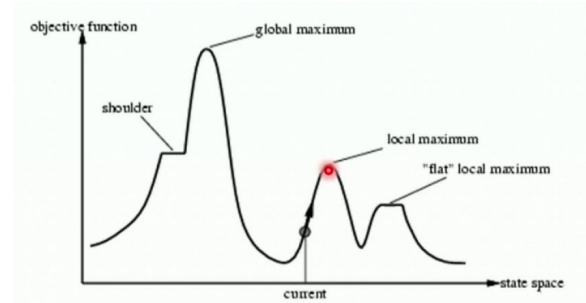
Hill Climbing is Called Greedy Local Search

Focus on Immediate Gains: Hill climbing's "greediness" comes from its strategy of always selecting the best immediate move that improves the current solution's value according to the objective function. It makes decisions based on the current, local information without considering the global context or potential long-term consequences.

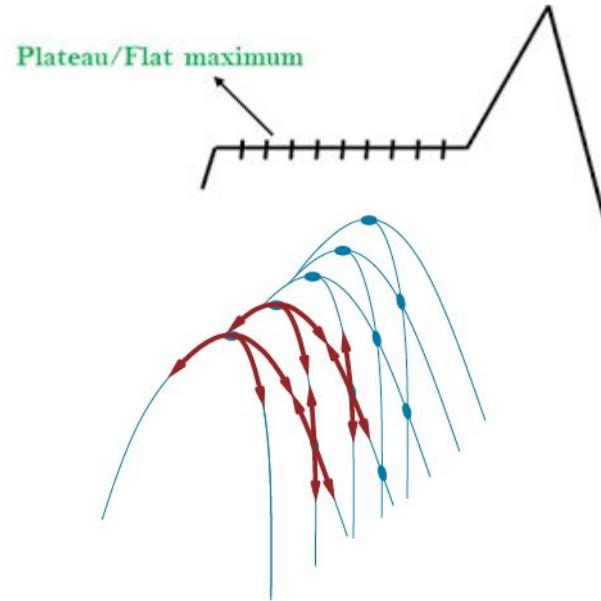
Limited Perspective: As a local search method, hill climbing only explores the immediate neighborhood of the current solution, further emphasizing its greedy approach by restricting its focus to nearby, small-scale improvements.

Drawbacks of Hill Climbing

1. Stuck in Local Optima



2. Plateau Problem



3. Ridges: A **ridge** is a narrow path of optimal or near-optimal solutions surrounded by regions of lower values.

Variants of Hill Climbing

- ◆ **Steepest-Ascent Hill Climbing**
- ◆ **Stochastic Hill Climbing**
- ◆ **First-Choice Hill Climbing:**
- ◆ **Random-Restart Hill Climbing**

Illustration:

Variant	Mountain Analogy	Outcome
Steepest-Ascent Hill Climbing	Check all nearby hills and move to the highest one. Repeat until no higher hills are nearby.	Likely to get stuck at a local maximum since it only considers the steepest local climb.
Stochastic Hill Climbing	Randomly check one nearby hill, move if it's higher. Continue picking random neighbors.	Some randomness helps in exploration, but you can still get stuck in a local optimum if the neighbors aren't better.
First-Choice Hill Climbing	Check nearby hills one by one, move as soon as you find a higher hill.	Faster than steepest-ascent, but may miss better hills since you only move on the first better option found.
Random-Restart Hill Climbing	Climb a hill to the peak, then restart from a new random point. Repeat until you find the highest peak.	Multiple random starting points give you a better chance of finding the global maximum after many attempts.
Greedy Hill Climbing	Always move to the immediate best neighbor (without checking all neighbors). No consideration for future moves.	Moves toward the best immediate hill without considering longer-term gains. Fast but very prone to getting stuck in local optima.

Steepest-Ascent Hill Climbing

Steepest-Ascent Hill Climbing is a variant of the hill climbing algorithm that seeks to address some of the limitations of basic hill climbing by systematically and comprehensively evaluating all possible moves (neighbors) before making a decision. This approach ensures that the algorithm always chooses the most significant improvement available in the current neighborhood, hence the name "steepest-ascent."

- **Basic Idea:**
 - The goal is to iteratively move towards the best solution by making the steepest possible ascent (or descent in the case of minimization problems) at each step.
 - Unlike the simpler "greedy" hill climbing that might choose the first neighbor that improves the objective function, steepest-ascent hill climbing evaluates all neighboring solutions and selects the one that offers the greatest improvement.

Step-by-Step Process of Steepest-Ascent Hill Climbing

Step 1: Start with an Initial Solution

- **Initial State Selection:**
 - As with other hill climbing algorithms, steepest-ascent hill climbing starts with an initial solution. This solution can be chosen randomly or based on some heuristic.

Step 2: Evaluate the Objective Function

- **Objective Function:**
 - The algorithm evaluates the objective function for the current solution to determine its fitness or quality. The objective function is the metric that the algorithm seeks to optimize (either maximize or minimize).

Step 3: Generate and Evaluate All Neighboring Solutions

- **Neighborhood Generation:**
 - The algorithm identifies all possible neighboring solutions of the current solution.
 - **Example in Numerical Optimization:** If the solution is represented by a vector of variables, a neighbor might be generated by slightly perturbing one of the variables.
- **Comprehensive Evaluation:**
 - **Evaluate Each Neighbor:** The algorithm calculates the objective function value for each neighboring solution.
 - **Compare Neighbors:** After evaluating all neighbors, the algorithm compares their objective function values to determine which one offers the steepest ascent (or descent).

Step 4: Select the Best Neighbor

- **Best Neighbor Selection:**
 - The neighbor with the highest improvement (for maximization problems) or the greatest reduction (for minimization problems) in the objective function is selected as the next solution.
- **No Improvement Handling:**
 - If no neighboring solution is better than the current solution, the algorithm recognizes that it has reached a local optimum, where no further improvements are possible within the current neighborhood.

Step 5: Move to the Neighboring Solution

- **Update the Current Solution:**
 - The algorithm moves to the selected neighboring solution, which becomes the new current solution. This move represents the steepest possible ascent in the objective function based on the local neighborhood.
- **Iterative Process:**
 - The process then repeats from the new solution, with the algorithm generating and evaluating its neighbors, selecting the best one, and moving again.

Step 6: Termination Criteria

- **Local Optimum Recognition:**
 - The algorithm terminates when it reaches a point where no neighboring solution improves the objective function. This situation indicates that the algorithm has reached a local optimum, and further exploration in the current neighborhood will not yield better results.
- **Alternative Termination Conditions:**
 - **Maximum Iterations:** The algorithm might stop after a predefined number of iterations.
 - **Objective Threshold:** The algorithm could also stop when the objective function value reaches a certain target or threshold.

Advantages of Steepest-Ascent Hill Climbing

Systematic Exploration:

- By evaluating all neighbors, steepest-ascent hill climbing avoids the risk of missing a good move due to a hasty decision, making it more reliable in finding the best possible local solution within the current neighborhood.

Efficiency in Smooth Search Spaces:

- This algorithm performs well in smooth, well-behaved search spaces where local improvements consistently lead toward the global optimum.

Fewer Local Optima Issues:

- While still prone to local optima, the systematic approach reduces the chances of getting stuck early in suboptimal regions of the search space.

Limitations of Steepest-Ascent Hill Climbing

- **Computational Cost:**
 - Evaluating all neighboring solutions can be computationally expensive, especially in high-dimensional search spaces where the number of neighbors is large.
- **Local Optima:**
 - Despite its systematic approach, steepest-ascent hill climbing can still get stuck in local optima, particularly in complex, multi-modal search spaces where the global optimum is not easily reachable from any local optimum.
- **No Global Perspective:**
 - The algorithm only considers local information and does not have any mechanism to escape local optima or explore more distant parts of the search space.

Stochastic Hill Climbing

Stochastic Hill Climbing is a variant of the hill climbing algorithm that introduces an element of randomness into the search process. Unlike traditional hill climbing methods, which deterministically select the best neighbor at each step, stochastic hill climbing randomly selects a neighbor to move to, with a preference for better neighbors. This randomness helps the algorithm explore the search space more broadly and potentially avoid getting stuck in local optima.

- **Basic Idea:**
 - Stochastic Hill Climbing modifies the standard hill climbing approach by introducing randomization into the selection of the next move. Instead of always choosing the best possible neighbor (as in steepest-ascent hill climbing), stochastic hill climbing may choose a neighbor that is not necessarily the best, based on a probabilistic criterion.
 - This approach helps the algorithm escape local optima by allowing it to explore less optimal regions of the search space that could potentially lead to better solutions in the long run.

Step-by-Step Process of Stochastic Hill Climbing

Step 1: Start with an Initial Solution

- **Initial State Selection:**
 - The algorithm begins with an initial solution, which can be selected randomly or based on a heuristic.

Step 2: Evaluate the Objective Function

- **Objective Function:**
 - The algorithm evaluates the objective function at the current solution. This function measures the quality of the solution and provides a basis for comparing neighboring solutions.

Step 3: Generate Neighboring Solutions

- **Neighborhood Generation:**
 - The algorithm identifies neighboring solutions by making small changes to the current solution.

Step 4: Select a Neighbor Stochastically

- **Stochastic Selection Process:**
 - Unlike deterministic hill climbing, which selects the best neighbor, stochastic hill climbing selects a neighbor randomly. However, this selection is often biased towards better neighbors—those that improve the objective function.
 - **Probability Distribution:** The likelihood of selecting a particular neighbor can be determined by a probability distribution. For instance, neighbors that offer greater improvements might be selected with higher probability, while less favorable neighbors are less likely but still have a chance of being selected.

Step 5: Move to the Selected Neighbor

- **Update the Current Solution:**
 - The algorithm moves to the selected neighboring solution. This solution becomes the new current solution for the next iteration.
- **Iterative Process:**
 - The process then repeats, with the algorithm generating new neighbors, selecting one stochastically, and moving to it.

Step 6: Termination Criteria

- **Termination Conditions:**
 - The algorithm can terminate under several conditions:
 - **No Improvement:** If after several iterations no significant improvement is found, the algorithm may stop.
 - **Maximum Iterations:** The algorithm stops after a predefined number of iterations.
 - **Time Limit:** The algorithm stops after a certain amount of computational time has passed.
 - **Objective Threshold:** The algorithm stops if the objective function reaches a desired target value.

Advantages of Stochastic Hill Climbing

Escaping Local Optima:

- The primary advantage of stochastic hill climbing is its ability to escape local optima. By allowing moves that are not strictly the best, the algorithm can explore regions of the search space that deterministic hill climbing might overlook.

Broad Exploration:

- The random selection process enables the algorithm to explore the search space more broadly, potentially leading to better overall solutions.

Simplicity and Flexibility:

- Stochastic hill climbing is relatively simple to implement and can be easily adapted to different types of optimization problems by adjusting the probability distribution for selecting neighbors.

Limitations of Stochastic Hill Climbing

Potentially Slower Convergence:

- Because the algorithm may sometimes choose suboptimal moves, it can take longer to converge to a solution compared to deterministic methods. This slower convergence is the trade-off for potentially escaping local optima.

No Guarantee of Global Optimum:

- Like other hill climbing methods, stochastic hill climbing does not guarantee finding the global optimum. It may still get stuck in local optima, especially if the search space is highly irregular or if the probability distribution is not well-tuned.

Dependence on Probability Distribution:

- The effectiveness of stochastic hill climbing can heavily depend on how the probability distribution is defined. Poorly chosen distributions may lead to ineffective exploration or excessive randomness, reducing the algorithm's efficiency.

First-Choice Hill Climbing

First-Choice Hill Climbing is a variant of the hill climbing algorithm designed to balance between the thoroughness of steepest-ascent hill climbing and the efficiency of simple hill climbing. In this approach, the algorithm randomly generates neighboring solutions and selects the first one that offers an improvement over the current solution. This method is particularly useful when the neighborhood of each solution is large, and evaluating all neighbors would be computationally expensive.

- **Basic Idea:**
 - First-Choice Hill Climbing combines elements of randomness and greediness. Instead of systematically evaluating all neighbors or choosing the best one from a pre-generated set, the algorithm evaluates neighbors one by one in a random order. It accepts the first neighbor that improves the objective function, which makes it faster than steepest-ascent hill climbing while still capable of escaping local optima more effectively than simple hill climbing.

Step-by-Step Process of First-Choice Hill Climbing

Step 1: Start with an Initial Solution

- **Initial State Selection:**
 - The algorithm starts with an initial solution, which can be chosen randomly or based on some heuristic.

Step 2: Evaluate the Objective Function

- **Objective Function:**
 - The algorithm evaluates the objective function at the current solution to determine its quality or fitness. The objective function quantifies how good or bad the current solution is with respect to the problem's goals.

Step 3: Generate and Evaluate Neighboring Solutions One by One

- **Neighborhood Generation:**
 - The algorithm generates neighboring solutions of the current solution by making small, specific changes. These changes define the neighborhood around the current solution.
- **Randomized Evaluation:**
 - **Random Selection:** The algorithm does not generate all neighbors at once. Instead, it generates neighbors one at a time in a random order.
 - **Immediate Evaluation:** Each time a neighbor is generated, the algorithm immediately evaluates its objective function value.

Step 4: Select the First Improving Neighbor

- **Immediate Selection:**
 - The algorithm selects the first neighbor that offers an improvement over the current solution. Once a better neighbor is found, the search stops evaluating other neighbors and moves to this new solution.
- **Greedy Acceptance:**
 - The acceptance criterion is greedy in nature: the algorithm only accepts moves that improve the objective function. This ensures that each move is a step towards a better solution.

Step 5: Move to the New Solution

- **Update the Current Solution:**
 - Once a better neighbor is found, the algorithm moves to this new solution. This new solution becomes the current solution for the next iteration of the algorithm.
- **Iterative Process:**
 - The process repeats from this new solution, with the algorithm generating new neighbors, evaluating them one by one, and moving to the first one that improves the objective function.

Step 6: Termination Criteria

- **Local Optimum Recognition:**
 - The algorithm terminates when it can no longer find a neighbor that improves the objective function. This indicates that a local optimum has been reached, where no further improvements are possible within the current neighborhood.
- **Alternative Termination Conditions:**
 - **Maximum Iterations:** The algorithm might stop after a predefined number of iterations.
 - **Objective Threshold:** The algorithm could also stop if the objective function value reaches a certain target or threshold.
 - **Time Limit:** The algorithm might stop after a certain amount of computational time has passed.

Advantages of First-Choice Hill Climbing

Efficiency:

- First-Choice Hill Climbing is more efficient than steepest-ascent hill climbing because it does not require evaluating all neighbors. By stopping as soon as an improvement is found, the algorithm reduces the computational burden, especially in large search spaces.

Escape from Local Optima:

- The randomized nature of the neighbor selection process helps the algorithm escape local optima. By not always selecting the globally best neighbor, the algorithm can explore more diverse areas of the search space.

Simplicity and Flexibility:

- The algorithm is straightforward to implement and can be easily adapted to various types of optimization problems. Its flexibility comes from the randomization in neighbor selection, which can be adjusted based on the problem's requirements.

Limitations of First-Choice Hill Climbing

Potential Suboptimal Choices:

- Because the algorithm selects the first improving neighbor rather than the best possible neighbor, it may make suboptimal moves. This could lead to slower convergence to the optimal solution or settling for a less-than-optimal solution.

Still Prone to Local Optima:

- Although more flexible than simple hill climbing, First-Choice Hill Climbing can still get stuck in local optima, particularly if the search space is highly irregular or contains many deceptive local peaks.

Dependence on Randomization:

- The performance of the algorithm can be sensitive to the randomization process. Poorly designed randomization may lead to inefficient exploration or excessive cycling between similar solutions.

Random-Restart Hill Climbing

Random-Restart Hill Climbing is a variant of the hill climbing algorithm that seeks to overcome one of the most significant limitations of basic hill climbing—getting stuck in local optima. This method enhances the standard hill climbing approach by running the algorithm multiple times from different random starting points in the search space. The idea is that if one run gets stuck in a local optimum, subsequent runs from different starting points may find better solutions, potentially leading to the global optimum.

- **Basic Idea:**
 - Random-Restart Hill Climbing is a meta-algorithm, meaning it wraps around the basic hill climbing algorithm. Instead of relying on a single run of hill climbing, which might get trapped in a local optimum, the algorithm runs hill climbing multiple times from different randomly chosen initial solutions. The best solution found across all these runs is then chosen as the final solution.

Step-by-Step Process of Random-Restart Hill Climbing

Step 1: Choose the Number of Restarts

- **Determining the Number of Restarts:**

- Before starting the algorithm, decide how many times the hill climbing algorithm will be restarted. This number can be fixed or adaptive, depending on the problem's complexity and the computational resources available.
- **Fixed Number of Restarts:** Predefine a specific number of restarts based on experience or trial-and-error.
- **Adaptive Restarts:** Continue restarting until a satisfactory solution is found or until computational resources (e.g., time) are exhausted.

Step 2: Start the First Hill Climbing Run

- **Initial Random Solution:**

- Randomly select an initial solution from the search space. This solution serves as the starting point for the first run of the hill climbing algorithm.

- **Run Hill Climbing:**

- Perform the standard hill climbing algorithm starting from this initial solution. The algorithm will iterate through generating and evaluating neighbors, selecting the best one, and moving to it until a local optimum is reached.

Step 3: Evaluate and Record the Result

- **Objective Function Evaluation:**
 - Once the hill climbing algorithm converges to a local optimum, evaluate the quality of this solution using the objective function.
- **Record the Best Solution:**
 - Keep track of the best solution found so far. If this solution is better than any previously recorded solutions, update the best solution.

Step 4: Repeat for Additional Restarts

- **Subsequent Random Solutions:**
 - For each subsequent restart, choose a new random initial solution from the search space.
- **Run Hill Climbing Again:**
 - Perform the hill climbing algorithm from this new starting point, allowing it to find another local optimum.
- **Evaluate and Record:**
 - After each restart, evaluate the resulting solution and update the best solution if necessary.

Step 5: Termination Criteria

- **Fixed Restarts:**
 - If the algorithm is using a fixed number of restarts, it will stop after the predefined number of runs. The best solution found across all runs is returned as the final solution.
- **Adaptive Restarts:**
 - If the algorithm is adaptive, it may stop when a satisfactory solution is found or when a predefined computational budget (e.g., time or iterations) is exhausted.

Advantages of Random-Restart Hill Climbing

- **Increased Probability of Finding Global Optimum:**
 - By running hill climbing multiple times from different starting points, Random-Restart Hill Climbing significantly increases the chances of finding the global optimum, even in complex search spaces with many local optima.
- **Flexibility:**
 - The algorithm can be adapted to different problem complexities by adjusting the number of restarts. It can be fine-tuned to balance between thoroughness and computational efficiency.
- **Simplicity:**
 - Random-Restart Hill Climbing is straightforward to implement. It leverages the simplicity of the basic hill climbing algorithm while adding a layer of robustness against local optima.

Limitations of Random-Restart Hill Climbing

Computational Cost:

- Running the hill climbing algorithm multiple times can be computationally expensive, especially if each run requires significant time or if many restarts are necessary to find a good solution.

No Guarantee of Global Optimum:

- While Random-Restart Hill Climbing increases the chances of finding the global optimum, it does not guarantee it. If the search space is vast or highly complex, even multiple restarts may not be sufficient to find the best solution.

Dependence on the Number of Restarts:

- The effectiveness of the algorithm heavily depends on the number of restarts. Too few restarts may lead to poor performance, while too many may result in unnecessary computational overhead.

Key Differences: A Summary

Aspect	Steepest-Ascent Hill Climbing	Stochastic Hill Climbing	First-Choice Hill Climbing	Random-Restart Hill Climbing
Neighbor Selection	Evaluates all neighbors and chooses the best one.	Randomly selects one neighbor and evaluates it.	Evaluates neighbors sequentially, moves to first better one.	Standard hill climbing, but restarts with a new random state if stuck.
Exploration Strategy	Exhaustively checks all neighbors before making a move.	Introduces randomness by evaluating one random neighbor.	Moves as soon as it finds a better neighbor, improving exploration.	Restarts from different random points to explore the space further.
Efficiency	Less efficient, as all neighbors are evaluated at each step.	More efficient, only one random neighbor is evaluated.	More efficient than steepest-ascent, only evaluates until an improvement is found.	Potentially inefficient due to multiple restarts but explores more broadly.
Convergence	Converges to a local maximum quickly, as the best move is always taken.	May converge more slowly as random neighbors are selected.	Converges faster than steepest-ascent, as it makes early moves when a better neighbor is found.	Converges eventually, but after multiple restarts, increasing the likelihood of finding a global maximum.

Handling Local Optima	Can get stuck in local optima, as it always chooses the steepest ascent.	May escape local optima, as worse neighbors might not always be avoided.	Can still get stuck in local optima, though faster escape due to early moves.	Escapes local optima by restarting with a new random initial state.
Memory Usage	Requires memory to evaluate all neighbors at each iteration.	Requires less memory, as only one neighbor is evaluated.	Requires less memory, only a few neighbors are evaluated sequentially.	Same as standard hill climbing, but memory usage increases with multiple restarts.
Pros	Guarantees the best local move in each iteration.	Introduces randomness, which improves exploration of the search space.	Faster convergence, more efficient than steepest-ascent.	Increases the chances of finding a global optimum by exploring multiple points in the search space.
Cons	Slower due to evaluating all neighbors, prone to getting stuck in local optima.	Might miss good moves as it only selects one random neighbor, still prone to local optima.	May not find the best neighboring solution, might still get stuck in local optima.	Computationally expensive due to multiple restarts, especially for large search spaces.
Best Used For	Problems where evaluating all possible moves is feasible and optimal local solutions are necessary.	Problems where randomness can help escape local optima, or when evaluating all neighbors is computationally expensive.	When a faster alternative to steepest-ascent is needed but evaluating all neighbors is not necessary.	Problems with complex landscapes and many local optima, where exploring many different parts of the search space is required.

Enforced Hill Climbing

Enforced Hill Climbing enhances the traditional hill-climbing algorithm by adding a mechanism to "force" progress in situations where the search might get stuck, such as local optima or plateaus. Here's how it works:

1. **Initial Step:**
 - The algorithm starts at a random initial state, just like traditional hill climbing, and begins exploring the neighboring states.
2. **Local Search Phase:**
 - It performs a local search to find an improved neighboring state with a better objective function value (higher in maximization or lower in minimization).
3. **Detection of Stagnation:**
 - If no neighboring state offers an improvement over the current state, the algorithm identifies that it has either hit a local optima or a plateau.
4. **Enforcement Mechanism:**
 - When no improvement is found, EHC enforces a change by **expanding the search horizon**. Instead of focusing only on immediate neighbors, it systematically **expands the search radius** by performing a **breadth-first search (BFS)** or a similar exploration technique.
 - This expansion continues until a state is found that offers an improvement over the current state.
5. **Recovery:**
 - Once an improving state is found through this enforced exploration, the algorithm moves to that state and resumes the traditional local hill climbing process.
6. **Termination:**
 - The algorithm terminates when it reaches a state from which no improvement can be enforced even after expanding the search.

Tabu Search

Tabu Search is a metaheuristic optimization algorithm used for solving combinatorial optimization problems. It was developed by Fred W. Glover in the late 1980s and is designed to escape local optima by allowing certain non-improving moves and preventing cycling back to previously visited solutions. Tabu Search is commonly applied in various domains, including scheduling, routing, and network design problems.

Key Concepts of Tabu Search

1. **Neighborhood Structure:**

Tabu Search relies on exploring the neighborhood of the current solution. A neighborhood is a set of candidate solutions that are generated by making small changes to the current solution. These changes could be simple modifications like swapping, inserting, or removing elements.

2. **Tabu List:**

The core idea of Tabu Search is to avoid revisiting the same solutions repeatedly. To achieve this, it maintains a **tabu list**, which stores recently visited solutions or moves for a certain period (known as the tabu tenure). Solutions or moves in the tabu list are forbidden or "tabu" during this period. The list helps prevent cycles and encourages exploration of new areas of the solution space.

3. **Aspiration Criteria:**

Although certain moves are classified as tabu, they can be allowed if they meet specific criteria known as **aspiration criteria**. The most common aspiration criterion is that if a move leads to a solution better than any seen so far, it is accepted even if it is in the tabu list. This prevents tabu restrictions from overly limiting the search.

4. **Intensification and Diversification:**

- **Intensification** refers to focusing the search on a promising region of the solution space to find a better solution.
- **Diversification** aims to explore new regions of the solution space to avoid local optima and discover new potential solutions. This is achieved by altering the neighborhood exploration strategy or resetting the search to an entirely different part of the search space.

5. **Objective Function:**

Tabu Search requires an objective function that evaluates the quality or cost of a solution. The algorithm aims to minimize or maximize this objective, depending on the problem.

6. **Termination Criteria:**

Tabu Search continues until a termination condition is met, such as a maximum number of iterations, a time limit, or no improvement in solution quality over a specified number of iterations.

Algorithm Steps

1. **Initialization:**
Start with an initial feasible solution, S_0 , and define the neighborhood structure, tabu list, and aspiration criteria.
2. **Neighborhood Exploration:**
Generate a set of candidate solutions by applying small changes (moves) to the current solution.
3. **Tabu List Check:**
Evaluate each candidate solution. If a move leads to a solution that is in the tabu list, discard it unless it meets the aspiration criteria.
4. **Selection of Best Move:**
Select the best non-tabu move (or tabu move meeting the aspiration criteria) and update the current solution.
5. **Update Tabu List:**
Add the move that led to the new solution to the tabu list and remove the oldest move if the tabu list is full (based on the tabu tenure).
6. **Repeat:**
Repeat the process of generating neighborhoods, selecting the best move, and updating the tabu list until the termination criteria are satisfied.

Advantages of Tabu Search

- **Escaping Local Optima:**
The tabu list helps Tabu Search avoid getting stuck in local optima, which is a common issue in many local search algorithms.
- **Flexible and Adaptable:**
It can be applied to a wide range of optimization problems and can handle constraints effectively through its flexible design.
- **Efficient Memory Usage:**
By storing only a limited amount of information (such as recent moves), the tabu list is efficient in terms of memory usage, making it scalable for large problems.

Limitations

- **Parameter Sensitivity:**
The performance of Tabu Search is highly dependent on parameters such as the size of the tabu list, tabu tenure, and the aspiration criteria. These parameters often need careful tuning for different problems.
- **Computational Cost:**
Since Tabu Search explores a wide range of solutions, the number of iterations can be large, leading to higher computational costs for complex problems.

Variants of Tabu Search

1. **Multi-Start Tabu Search:**

Restarting the search multiple times from different initial solutions to explore the solution space more broadly.

2. **Reactive Tabu Search:**

Dynamically adjusting the tabu tenure based on the search progress to better balance between intensification and diversification.

3. **Parallel Tabu Search:**

Running multiple instances of Tabu Search in parallel to explore different regions of the solution space simultaneously.

Simulated Annealing

Origin of Simulated Annealing

Simulated Annealing (SA) takes its inspiration from a physical process used in metallurgy called **annealing**. In this process, a material is heated to a high temperature and then cooled slowly to remove defects and achieve a more stable structure.

- **Annealing in Metallurgy:**

- The material is initially heated to a temperature where its atoms move freely. At this stage, the material is in a high-energy state.
- It is then cooled gradually, allowing the atoms to settle into a more ordered, low-energy configuration.
- If cooled too quickly, the atoms may become trapped in a disordered, higher-energy state, much like a local minimum in optimization problems.

This process in metallurgy has a direct analogy to **optimization** problems in artificial intelligence.

Metaphor to Optimization

The key idea behind Simulated Annealing is to allow the search process to accept not only better solutions but also worse ones, especially at higher temperatures. This ability to accept worse solutions helps the algorithm escape local optima, which is critical for finding the global optimum in complex search spaces.

- **High Temperature Phase:**
 - Initially, the system is at a high temperature, which means it can move freely across the solution space. In this phase, the system can jump between various states, including those that are worse than the current solution.
 - This helps prevent the search from getting stuck in a local minimum early on.
- **Cooling Phase:**
 - As the algorithm progresses, the temperature is gradually lowered, reducing the probability of accepting worse solutions.
 - The system becomes more selective and focuses on refining the solution, similar to how atoms settle into a low-energy configuration during the cooling phase of annealing.

By allowing the search to explore a wide variety of solutions in the beginning and then slowly reducing the likelihood of accepting worse solutions, SA balances exploration and exploitation.

Core Components of Simulated Annealing

4.1 Energy Function (Cost/Objective Function)

The **Energy Function** (often called the **Cost Function** or **Objective Function**) represents the quality or fitness of a solution.

- **Definition:**
 - The energy function assigns a value (or "cost") to each possible solution in the search space.
 - Lower energy values represent better solutions, and higher energy values represent worse solutions.
- **Role in SA:**
 - SA uses the energy function to evaluate how good or bad a solution is. The algorithm attempts to minimize this function, just as a physical material seeks to minimize its energy state during annealing.
- **Example:**
 - In the Travelling Salesman Problem (TSP), the energy function could be the total distance traveled. The goal would be to minimize this distance.

4.2 Temperature (T)

The **Temperature** (T) controls the algorithm's willingness to accept worse solutions.

- **Initial High Temperature:**

- At the start of the algorithm, the temperature is high, which means that the system can explore freely by accepting both better and worse solutions.
- The probability of accepting a worse solution is determined by the equation:

$$P(\Delta E) = \exp \left(\frac{-\Delta E}{T} \right)$$

where ΔE is the difference between the current solution and the new solution.

- If ΔE is positive (i.e., the new solution is worse), there is still a chance that the solution will be accepted, especially when T is high.
- **Decreasing Temperature:**
 - As the temperature decreases, the algorithm becomes less likely to accept worse solutions. Eventually, when the temperature is near zero, the system only accepts better solutions.
- **Intuition:**
 - At high temperatures, the algorithm explores more of the solution space (including suboptimal solutions).
 - At low temperatures, the algorithm focuses on refining the best solution found so far.

4.3 Cooling Schedule

The **Cooling Schedule** determines how the temperature is decreased over time.

- **Cooling Function:**

- The cooling schedule specifies the rate at which the temperature decreases as the algorithm progresses.
- Common cooling schedules include:

- **Linear Cooling:** $T = T_0 - \alpha k$, where k is the iteration number.

- **Exponential Cooling:** $T = T_0 \cdot \alpha^k$, where $\alpha < 1$ is a constant.

- **Logarithmic Cooling:** $T = \frac{T_0}{\log(1+k)}$, where k is the iteration number.

- **Choosing the Right Cooling Schedule:**

- A **slow cooling schedule** provides more time for the algorithm to explore, increasing the chances of finding a global optimum but at the cost of longer runtime.
 - A **fast cooling schedule** may lead to quicker convergence, but the system might miss the global optimum and get stuck in a local minimum.